Mini-projet : Mise en place d'un système de recommandation

Le but de ce mini-projet est de mettre en place un système de recommandation qui se base sur les algorithmes de filtrage collaboratif et de filtrage basé sur le contenu. En bonus, vous pouvez y ajouter une brique RAG et des pratiques MLOps (voir « Pour aller plus loin », page 2).

Pour cela, vous devez choisir un jeu de donnée

- Découvrir les données (qu'est-ce qu'il y a dedans, potentiel d'utilisation)
- Nettoyer éventuellement les données (choisir un sous ensemble pertinent pour l'objectif choisi)
- Eventuellement enrichir les données par d'autres jeux de données complémentaires
- Construire le jeu de données à traiter
- Choisir des algorithmes et appliquez-les
- Evaluer les performances avec des métriques choisies
- Interpréter les résultats
- Le cas échéant reboucler si les résultats ne sont pas conformes aux attentes
- Comparer les résultats obtenus avec les différentes approches utilisées
- Donner votre retour d'expérience sur les approches utilisées, les problèmes rencontrés et comment vous les avez résolus (ou pas).

Organisation du travail : par groupes de 2 personnes

Restitution du travail : le projet doit être déposé sur moodle. Un lien vers un git est aussi possible. Le rendu devra comprendre :

- Un petit rapport décrivant la problématique, les jeux de données choisis et les résultats obtenus.
- Une archive contenant le jeu de données et le code associé
- Une présentation orale

Quelques liens vers des jeux de données publics :

- https://cseweb.ucsd.edu/~jmcauley/datasets.html
- https://github.com/caserec/Datasets-for-Recommender-Systems

Pour aller plus loin (optionnel)

Les deux extensions ci-dessous sont facultatives et indépendantes : chacune constitue un plus dans l'évaluation. Le cœur du projet reste le filtrage collaboratif et le filtrage basé sur le contenu.

RAG (optionnel). Ajouter une brique RAG (Retrieval-Augmented Generation) n'est pas obligatoire, mais c'est un plus. Par exemple, indexez les descriptions et les avis des articles dans une base vectorielle, puis demandez à un LLM d'expliquer chaque recommandation en citant ces textes, ou de répondre à une demande comme « un film léger à voir en famille ». Si votre jeu de données porte sur des films, l'API de TMDB fournit les synopsis : avec MovieLens, la colonne tmdbId du fichier links.csv donne l'identifiant de chaque film.

Ressources pour le RAG :

- Exemple du cours : le notebook Lab_RAG.ipynb (FAISS, LLM local avec Ollama)
- API de films TMDB, gratuite pour un usage non commercial en citant TMDB : https://developer.themoviedb.org/docs/getting-started
- Autre API de films, OMDb : https://www.omdbapi.com/
- RAG pas à pas avec un LLM local (LlamaIndex + Ollama) : https://developers.llamaindex.ai/python/framework/getting_started/starter_example_local/
- Exemple plus complet (Hugging Face Cookbook) : https://huggingface.co/learn/cookbook/advanced_rag

MLOps (optionnel). Les pratiques MLOps sont elles aussi optionnelles et constituent un autre plus. Vous pouvez suivre vos expériences (paramètres, métriques, modèles) avec MLflow, exposer vos recommandations via un serveur et ses endpoints (par exemple FastAPI : `GET /recommend?user_id=42`), puis écrire un Dockerfile pour servir le tout en local. Indiquez dans le README comment lancer le serveur et le tester.

Ressources pour le MLOps :

- Exemple du cours : le projet ChurnGuard du cours MLOps (MLflow, FastAPI, Docker)
- Suivi d'expériences avec MLflow : https://mlflow.org/docs/latest/ml/tracking/quickstart/
- Tutoriel FastAPI : https://fastapi.tiangolo.com/tutorial/
- Dockerfile pour une application FastAPI : https://fastapi.tiangolo.com/deployment/docker/
- Premiers pas avec Docker : https://docs.docker.com/get-started/